TASKFLOW

TASK MANAGEMENT WEB APPLICATION

MINOR PROJECT REPORT

Submitted by	Nikhat Siddekee
Course	B.Tech in Computer Science and Engineering
Project Guide	Pratham Saxena
Academic Year	2026
Report Date	8 November 2026

Developed using HTML5, CSS3, JavaScript and Browser Local Storage

CERTIFICATE

This is to certify that **Nikhat Siddekee**, a student of B.Tech in Computer Science and Engineering, has completed the minor project titled **“TaskFlow: Task Management Web Application”** under the guidance of **Pratham Saxena**.

This report describes the project work and the features documented for the TaskFlow application.

Project Guide: Pratham Saxena
Student: Nikhat Siddekee
Date: 24 October 2026

ACKNOWLEDGEMENT

I would like to thank my project guide, Pratham Saxena, for the guidance and feedback provided during the development of this project. I also thank everyone who supported me while planning, developing, and documenting TaskFlow.

ABSTRACT

TaskFlow is a browser-based task management application developed using HTML, CSS, and JavaScript. It helps users create and organize tasks, assign priorities and due dates, update task details, and track whether work is pending or complete. The application includes a dashboard showing task totals, search, status and priority filters, sorting options, and visual highlighting for overdue pending tasks. Browser Local Storage retains tasks after the page is refreshed in the same browser profile.

1. INTRODUCTION

Managing assignments and everyday work can be difficult when tasks, deadlines, and priorities are not organized. TaskFlow provides a single browser interface to record tasks and track progress. Users can add task details, set a priority and due date, find tasks using search and filters, and mark tasks as complete or pending.

1.1 Problem Statement

Without an organized task list, users may forget deadlines or have difficulty identifying which tasks need attention first. TaskFlow addresses this problem by displaying task details, status, priority, and due dates together.

1.2 Project Objective

The objective is to develop a responsive task management application using HTML, CSS, and JavaScript that enables users to create, view, edit, delete, search, sort, filter, and track tasks through a clear interface.

1.3 Project Scope

The project focuses on client-side task management in a web browser. Task information is stored locally in the browser using Local Storage. The current implementation does not include user accounts, a remote database, or automatic synchronization between devices.

2. TECHNOLOGY STACK

Technology	Purpose in TaskFlow
HTML5	Structures the page, dashboard, task form, and task list.
CSS3	Styles the interface and provides responsive layouts for desktop, tablet, and mobile screens.
JavaScript	Implements task operations, validation, search, filters, sorting, dashboard statistics, and DOM updates.
Browser Local Storage	Stores the task array in the current browser profile so it can be restored after refresh.
Git and GitHub	Provides version control and public repository hosting for the source code and documentation.

3. APPLICATION FEATURES AND REQUIREMENTS

3.1 Functional Requirements

• Create tasks with a title, description, priority, and due date.
• View tasks and their status, priority, and due date.
• Edit existing task details and delete tasks after a confirmation prompt.
• Mark tasks as completed or pending.
• Search by task title or description; filter by status and priority.
• Sort tasks by due date, priority, or creation time.
• Display total, pending, completed, and overdue task counts.
• Validate required fields and show an empty state when no tasks match.
• Save and restore tasks using browser Local Storage.

3.2 Non-functional Requirements

The interface is designed to be responsive and usable on desktop, tablet, and mobile-sized screens. The application runs in a modern browser without requiring a separate server or database installation. Task data remains local to the browser profile and is not automatically synchronized to another device.

4. APPLICATION FEATURES

4.1 Task Management

Users can create tasks with a title, description, priority, and due date. Existing tasks can be edited or deleted. A confirmation prompt is shown before deletion.

4.2 Completion Status and Dashboard

Tasks can be marked completed or pending. The dashboard displays total, pending, completed, and overdue task counts. Statistics are recalculated when the task list changes.

4.3 Search, Filters, and Sorting

Users can search by task title or description, filter by completion status and priority, and sort by due date, priority, or creation time.

4.4 Overdue Indicator and Validation

A pending task with a due date earlier than the current date is identified as overdue and receives a visible label and highlighted card. Required fields and priority values are checked before a task is saved.

5. APPLICATION WORKFLOW

1. The user opens **index.html** in a web browser.
2. The user enters a title, description, priority, and due date.
3. JavaScript validates the form and adds a new task or updates an existing task.
4. The task is rendered in the task list.
5. The user can search, filter, sort, complete, edit, or delete tasks.
6. Dashboard counts update and task data is saved to Local Storage.
7. On a later page load or refresh, saved tasks are restored.

6. USER INTERFACE AND IMPLEMENTATION

The interface contains a header, dashboard cards, task form, search field, filter and sort controls, and task cards. The dashboard summarizes progress. The form collects task information. Search and filter controls help users locate tasks, while task cards display the title, description, priority, due date, status, and available actions.

CSS media queries adapt the dashboard, form, filters, and action buttons to smaller screens. JavaScript uses helper functions to load and save tasks, create DOM elements, render filtered tasks, and update dashboard statistics.

6.1 Local Storage and Data Handling

The task array is converted to JSON text before it is saved in Local Storage using the browser storage API. When the application loads, saved text is read and parsed back into JavaScript objects. The implementation handles storage read/write errors and checks that loaded data is an array. Local Storage is specific to the browser profile and device; it does not provide online account synchronization.

6.2 Project Links

GitHub Repository: https://github.com/Nikhat-codes/TaskFlow-Minor-Project

Live Demo: https://nikhat-codes.github.io/TaskFlow-Minor-Project/

7. SCREENSHOTS OF THE APPLICATION

The following figures are screenshots of the TaskFlow interface included with the project submission. They show the dashboard, task form, task list with its controls, and responsive mobile layout.

7.1 Main Dashboard

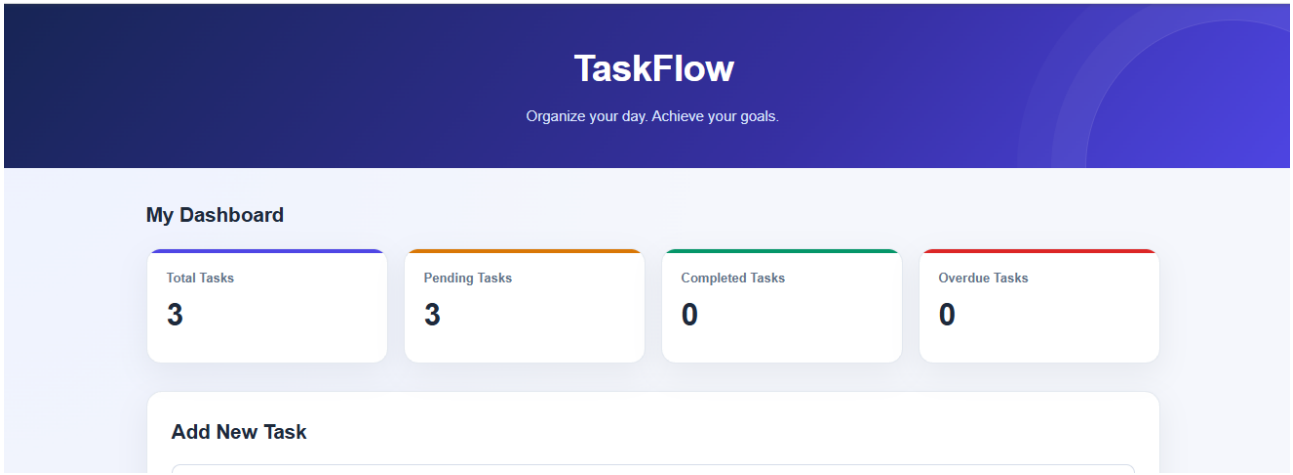


Figure 1. Main dashboard showing total, pending, completed, and overdue task summary cards.

7.2 Task Creation Form

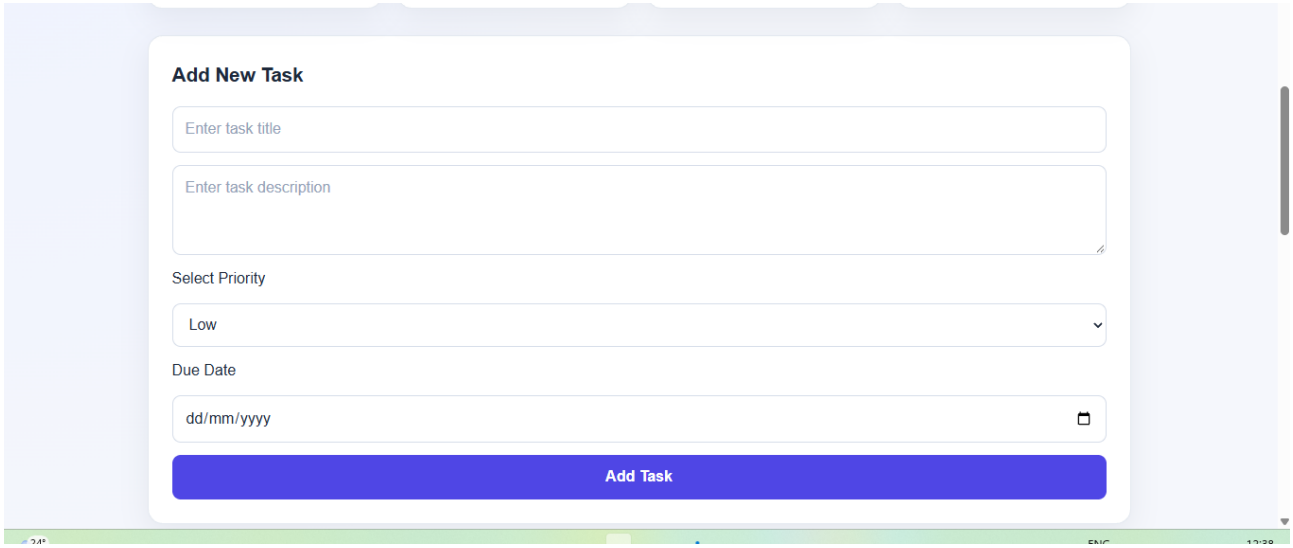


Figure 2. Task creation form with title, description, priority, and due-date fields.

7.3 Task List, Search, Filters, and Sorting

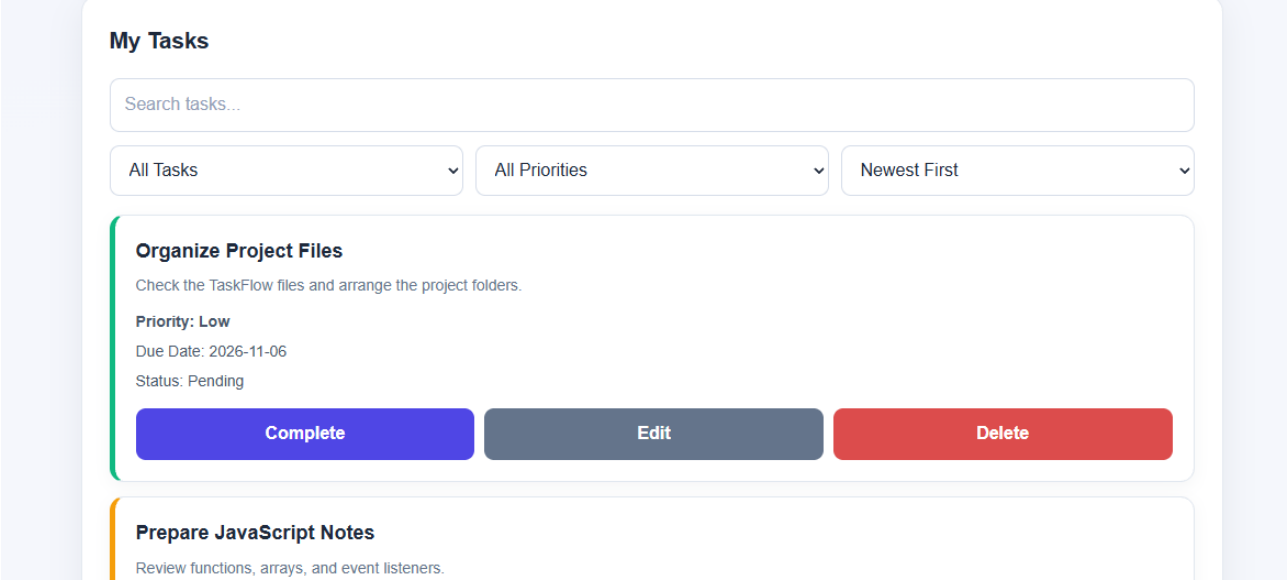


Figure 3. Task list showing task details, status, priority, search, filters, sorting, and task actions.

7.4 Responsive Mobile View

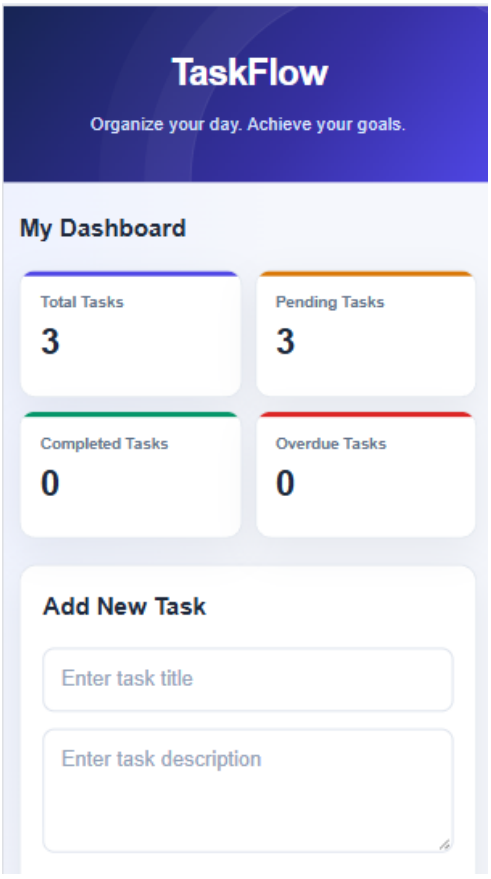


Figure 4. Responsive layout on a mobile-sized screen.

8. TESTING AND RESULTS

The following test cases and outcomes are recorded in the project testing checklist.

ID	Test case	Expected result	Result
TC-01	Add a task with valid details	Task appears in the list and dashboard total updates.	**Pass**
TC-02	Submit with missing required details	Incomplete task is not added.	**Pass**
TC-03	Edit a task	Updated details appear in the task list.	**Pass**
TC-04	Delete a task	Confirmation is requested before deletion.	**Pass**
TC-05	Complete or reopen a task	Task status and dashboard totals update.	**Pass**
TC-06	Search for a task	Matching tasks appear.	**Pass**
TC-07	Filter by status or priority	Only matching tasks appear.	**Pass**
TC-08	Sort the task list	Tasks appear in the selected order.	**Pass**
TC-09	Refresh the page	Previously saved tasks load again.	**Pass**
TC-10	Check an overdue pending task	Task is counted and visibly marked overdue.	**Pass**
TC-11	View on a mobile-sized screen	Layout fits the narrow screen.	**Pass**

9. CHALLENGES AND SOLUTIONS

Retaining tasks after refresh: Browser Local Storage is used to save and reload the task list.
Identifying overdue tasks: The application compares each pending task's due date with the current date and displays an overdue count and visual indicator.
Organizing multiple tasks: Search, filters, sorting, priorities, and status labels help users find and organize tasks.

10. LIMITATIONS AND FUTURE ENHANCEMENTS

The current application stores data locally in the browser and does not provide user accounts, cloud synchronization, or multi-device access. Possible future improvements include optional user authentication, cloud backup, recurring tasks, reminders and notifications, categories and tags, a calendar view, and export of task data.

11. CONCLUSION

TaskFlow demonstrates how HTML, CSS, JavaScript, and browser Local Storage can be combined to build a responsive task management application. It provides core task operations, dashboard statistics, search, filtering, sorting, overdue indicators, and persistent browser storage. The project provided practical experience with front-end development, responsive design, browser storage, and user-interface implementation.

12. REFERENCES

• MDN Web Docs — HTML documentation: https://developer.mozilla.org/docs/Web/HTML
• MDN Web Docs — CSS documentation: https://developer.mozilla.org/docs/Web/CSS
• MDN Web Docs — JavaScript documentation: https://developer.mozilla.org/docs/Web/JavaScript
• MDN Web Docs — Web Storage API: https://developer.mozilla.org/docs/Web/API/Web_Storage_API
• Project implementation files included in the **source_code** folder.

Project: TaskFlow — Task Management Web Application
Student: Nikhat Siddekee
Course: B.Tech in Computer Science and Engineering
Project Guide: Pratham Saxena